

Lecture 20

MVU II

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Aside: Text Field in Elm

- `input` represents an *HTML text field*:

```
import Html exposing (input)
import Html.Attribute exposing (value)
import Html.Events exposing (onInput)
```

```
input  : List (Attribute m) -> List (Html m) -> Html m
value  : String -> Attribute m
onInput : (String -> m) -> Attribute m
```

Aside: Text Field in Elm

```
type Msg =
    -- ...
    | MsgChangeText String -- MsgChangeText carries new text

-- ...
div []
    -- Elm will pass the new content to `MsgChangeText`
    -- and will pass the result to `onInput`;
    -- `value` contains displayed text in textbox
    [ input [ value model.text, onInput MsgChangeText ] []
    ]
```

Fetching Data Online

```
import Http

type Msg
    = MsgGotJoke (Result Http.Error String) -- Like Maybe, but
    error
    -- also returns something
```

Aside: Result

```
type Result error value
  = Ok value
  | Err error
```

- **Ok** means computation succeeded, result is `value`
- **Err** if there is some error, error message/type is `error`
- Process using pattern matching

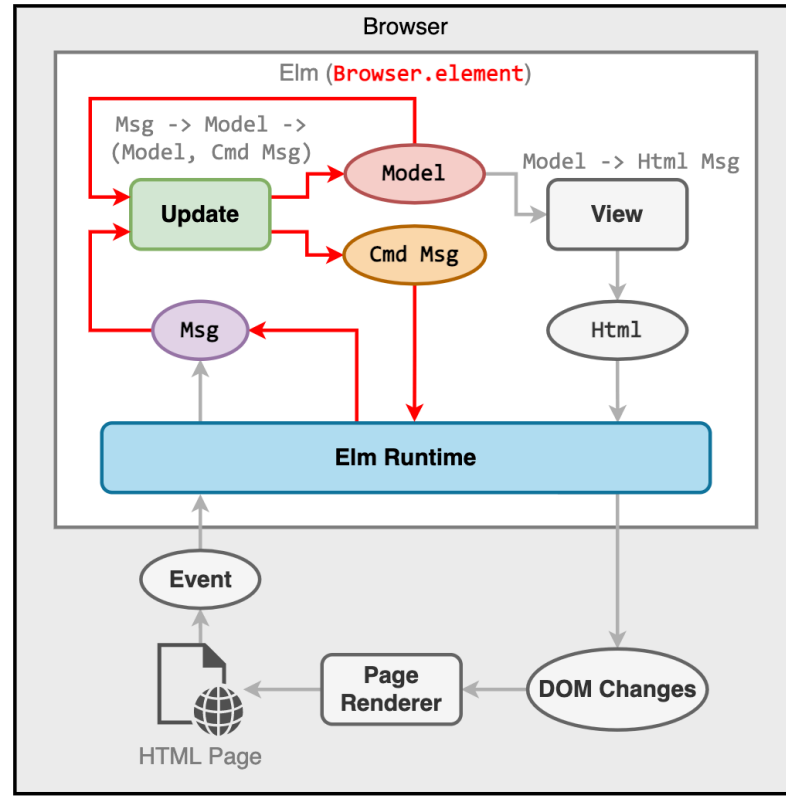
Fetching Data Online

```
Http.get : { url : String, expect : Expect m } -> Cmd m
```

```
getJoke: Cmd Msg
getJoke =
  -- Sends a GET request to specified URL
  Http.get
    { url = "https://v2.jokeapi.dev/joke/programming?format=txt"
    , expect = Http.expectString MsgGotJoke
    }
}
```

What is a `Cmd m`?

Commands



Running Commands

- Option 1: Via `init` (on program start)

```
init : () -> (Model, Cmd Msg)      -- Now (1) a function and (2)
returns a tuple
init _ = (LoadingJoke, getJoke)    -- getJoke is executed on
program start
```

- Option 2: Via `update` (reaction to a `Msg`)

```
update : Msg -> Model -> (Model, Cmd Msg)  -- Now returns a
tuple
```

Fetching Data Online

```
type Model
  = LoadingJoke          -- State when data is being fetched
  (start)
  | ShowingJoke String  -- State when data fetch succeeds
  | ErrorPage String    -- State when data fetch fails

type Msg = MsgGotJoke (Result Http.Error String)
```

Fetching Data Online

```
init : () -> (Model, Cmd Msg)    -- Now (1) a function and (2)
returns a tuple
init _ = (LoadingJoke, getJoke)  -- getJoke is executed on program
start
```

Fetching Data Online

```
update : Msg -> Model -> (Model, Cmd Msg)  -- Now returns a tuple
update msg model =
    case msg of
        MsgGotJoke result ->
            case result of
                Ok joke -> (ShowingJoke joke, Cmd.none)
                Err err -> (ErrorPage (Debug.toString err),
                    Cmd.none)

view : Model -> Html m      -- Same as before
view model =
```

```
case model of
```

```
  LoadingJoke -> text "Loading joke..."
```

```
  ShowingJoke joke -> text joke
```

```
  ErrorPage error -> text error
```

Fetching Data Online

```
import Browser

subscriptions : Model -> Sub Msg
subscriptions _ = Sub.none -- Ignore for now; not needed

main = Browser.element -- Browser.element instead of
Browser.sandbox
  { init = init
  , update = update
  , subscriptions = subscriptions
  , view = view }
```

Self-Check Assignment

Task: Add a URL textbox and a fetch button

- Do not fetch data on program start

Practice: Dispatching messages on text input

Task: Create an MVU web app that has two textboxes A and B on screen with text saying "Please enter integers"

- Whenever text is entered into either A or B:
 - If text in A and B can be parsed by `String.toInt`:
 - Replace text on screen with sum
 - Else:
 - Show "Please enter integers on screen"

Demo: Dispatching messages on text input

More practice: Dynamic DOM updates

- New counter button: Creates new counter
 - Counters must be independent of each other
- For each counter:
 - Count text: Shows current count
 - + button: Increments count by 1
 - - button: Decrements count by 1
 - Reset button: Sets count to 0
 - Delete button: Removes counter

Demo: Dynamic DOM updates